

**An End-to-End ETL Pipeline for Brazilian E-Commerce Retail Analytics:
Automated Data Ingestion, Quality Assurance, and Business Intelligence
Using Apache Airflow, PostgreSQL, and Power BI**

Author

Renato Silva.
Sandra Raj Pattuvakkaran.
Sudarsh Mekkampurath Sajeev.

Module Name

Data Engineering.

Submitted To

Prof. Dr. Nor Azizah Hitam

Submission Date

12 of June of 2026.

1. Project Overview & Objective:

This project developed a data engineering pipeline for the Olist Brazilian E-commerce datasets available at Kaggle. The datasets included approximately 100,000 real orders made between 2016 and 2018 and consisted of nine inter-related tables.

The objective of the project was to build an actual ETL pipeline to decide what to do with missing values, how to create connections between tables, and, most importantly, how to automate the entire process so the user does not need to click a button each time they want to execute the pipeline. The three main objectives that directed the project were:

- Build a complete pipeline that pulls in, cleans, and populates all nine tables into a PostgreSQL database with correct relationships between the tables.
- Automate the above pipeline using Apache Airflow, so it runs on a predetermined schedule and tracks errors.
- Connect the clean database to Power BI and provide dashboards to display delivery times, revenue trends per geographic location, the top-ranked sellers, and the average score of customer reviews.

2. Problem Statement & Methodology

Olist's raw data consists of nine separate CSV files. There is no relationship defined between these files. As a result, none of the data alone will allow you to ask or calculate answers to even one business question. In order to determine delivery time, you must join together orders to order items to the product. To analyze revenue by region, you must link your customers to their respective locations. Although the data exists, it is unusable in its current state.

The engineering problem is to develop a pipeline that takes the source files and automates the cleanup, creates the appropriate relationship(s), and formats the files so you can execute reliable queries. The additional constraint to the above is that the pipeline should run automatically and track failures with precision, rather than simply executing once.

To develop the methodology for this project, I followed the traditional ETL model with a developer/test split introduced to verify functionality:

- Extract the raw CSV files utilizing the Kaggle API.
- Explore the data to find quality issues.
- Transform by eliminating nulls, duplicates, type-mismatched fields, etc., and eliminating missing foreign key references.
- Split 75 / 25 as development and test sets centered around the orders table.
- Load the development set into PostgreSQL with a 3rd NF design.
- Validate with DAG execution using Apache Airflow and integrity tests written in Python.
- Test by validating with DAG execution using Apache Airflow and integrity tests written in Python against the test set.

Developer/Test split replicates how production pipelines are verified: a new pipeline is created, tested, and tweaked on one batch of data, then verified on another batch it has never processed before.

3. Data Collection & Preparation

We utilized the Kaggle API programmatically to collect our data. The pipeline connects to the API, collects the dataset, and unzips all nine files into a raw data directory. Environment issues surfaced here: Our network utilizes an SSL proxy that prevented standard pip installs and API calls. I resolved this issue by using the `--trusted-host` flag to bypass certificate verification. After loading all of the files into Python, during the exploration portion of the process, there were four categories of quality issues with the data found:

Table	Issue	Decision	Reason
orders	2,965 null delivery dates	Preserved	Represent undelivered orders valid data.
products	610 missing category names	Filled with 'unknown'	Category must exist for FK constraint
products	2 missing dimensions	Filled with median	Median avoids skew from outliers
reviews	87,656 missing titles	Filled with 'No title'	Most customers skip written feedback
reviews	58,247 missing messages	Filled with 'No comment'	Same reason as above
reviews	Duplicate review IDs	Keep first, drop rest	Bug in Olist's review submission system
category_translation	Missing categories (pc_gamer, etc.)	Added programmatically	Required for FK integrity with products

4. ETL Pipeline & Storage Design

The pipeline executes in a sequential manner: extracting from Kaggle, transforming within Python, loading into PostgreSQL. Apache Airflow manages both the execution order and scheduling of the pipeline. Below is the architecture flow illustrating the entire process:

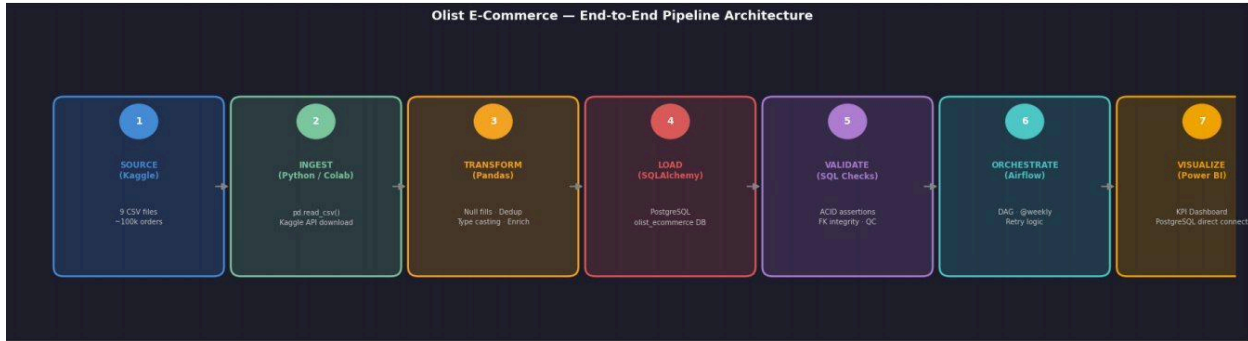


Figure 1: End-to-End Pipeline Architecture

Below is a representation of how the PostgreSQL schema was designed in 3NF. Each data field resides in exactly one table and is linked to other tables through primary and foreign keys. The schema contains two types of tables: reference tables (products, sellers, geolocation, category translations) and transaction tables (orders, customers, order items, payments, reviews). All reference tables are populated first because transaction tables contain foreign keys referencing them.

Important schema decisions:

- Order items utilize a composite primary key (`order_id` + `order_item_id`) since neither column is unique on its own, as one order may have multiple items associated with it.
- Payments utilize a composite primary key (`order_id` + `payment_sequential`) since one order can be split among multiple payment options.
- Geolocation has no primary key since multiple lat/lng coordinates exist for different zip-code prefixes.

Below is an entity-relationship diagram representing all primary and foreign-key relationships in the complete schema:

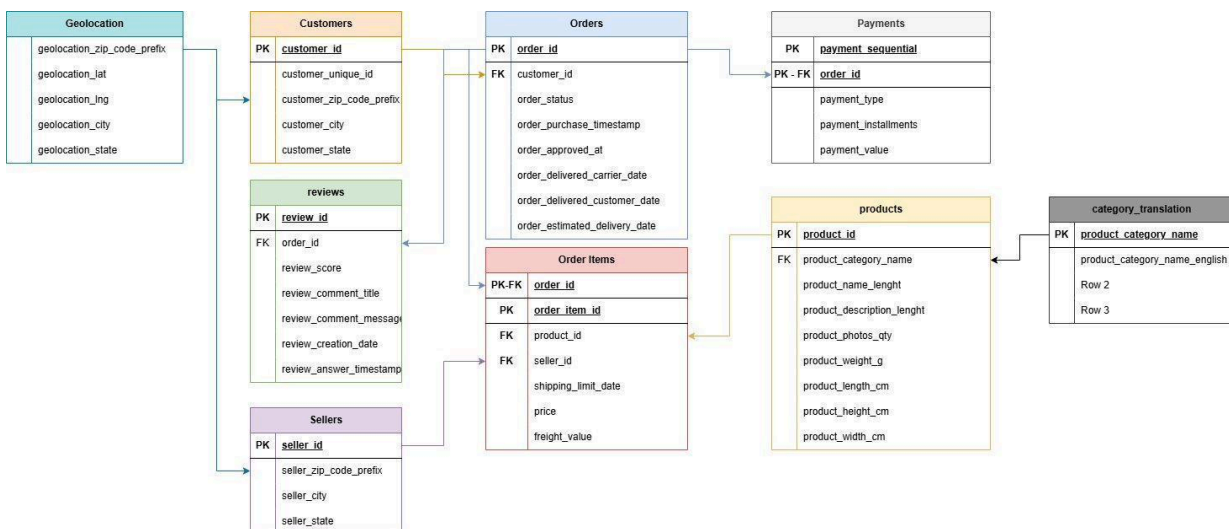


Figure 2: Entity-Relationship Diagram — olist_ecommerce database

The screenshot below confirms all nine tables were successfully created and populated in PostgreSQL 18:

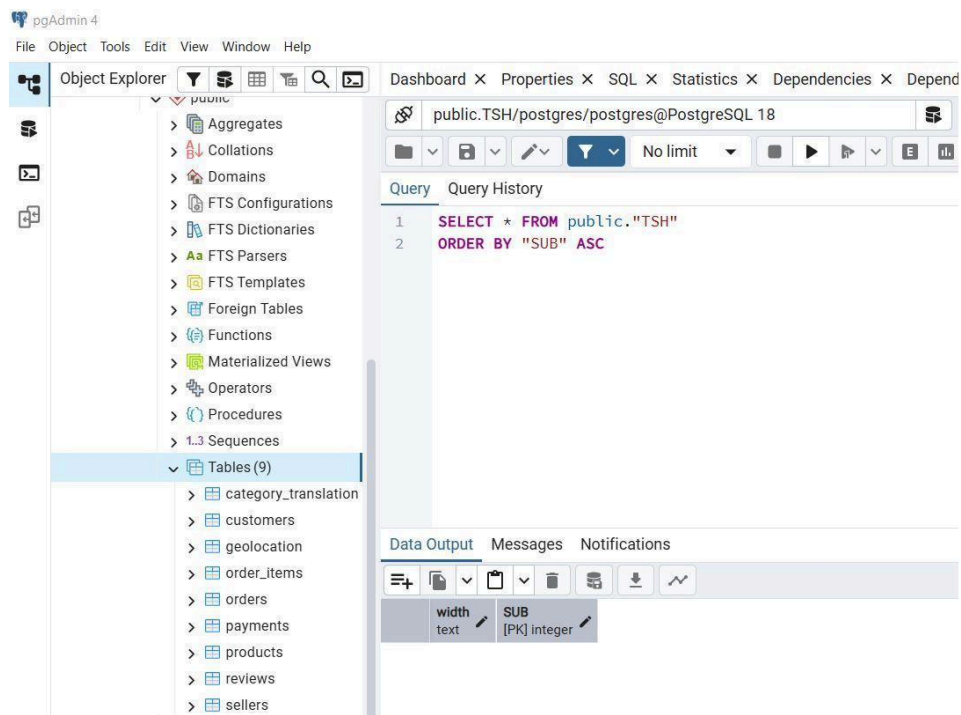


Figure 3: PostgreSQL pgAdmin showing all 9 tables in the olist_ecommerce database

5. Orchestration & Quality Assurance

An Apache Airflow DAG orchestrates the pipeline with four tasks running in sequence: extract, clean, validate, and notify. The DAG runs on a daily schedule with one automatic retry per task before marking a failure.

The Airflow DAG interface showing the pipeline structure:

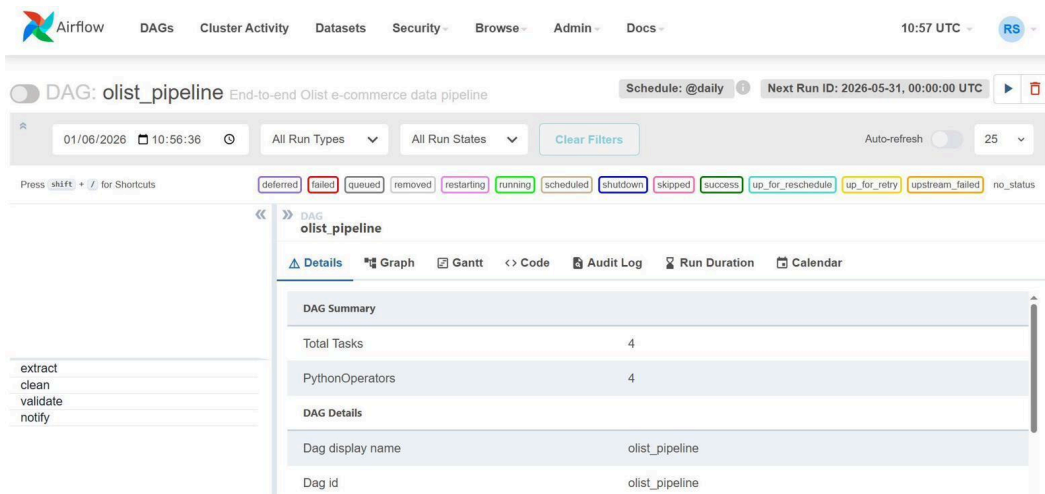


Figure 4: Airflow DAG `olist_pipeline` — details view showing 4 `PythonOperator` tasks

The screenshot below confirms all four tasks were completed successfully in the most recent run:

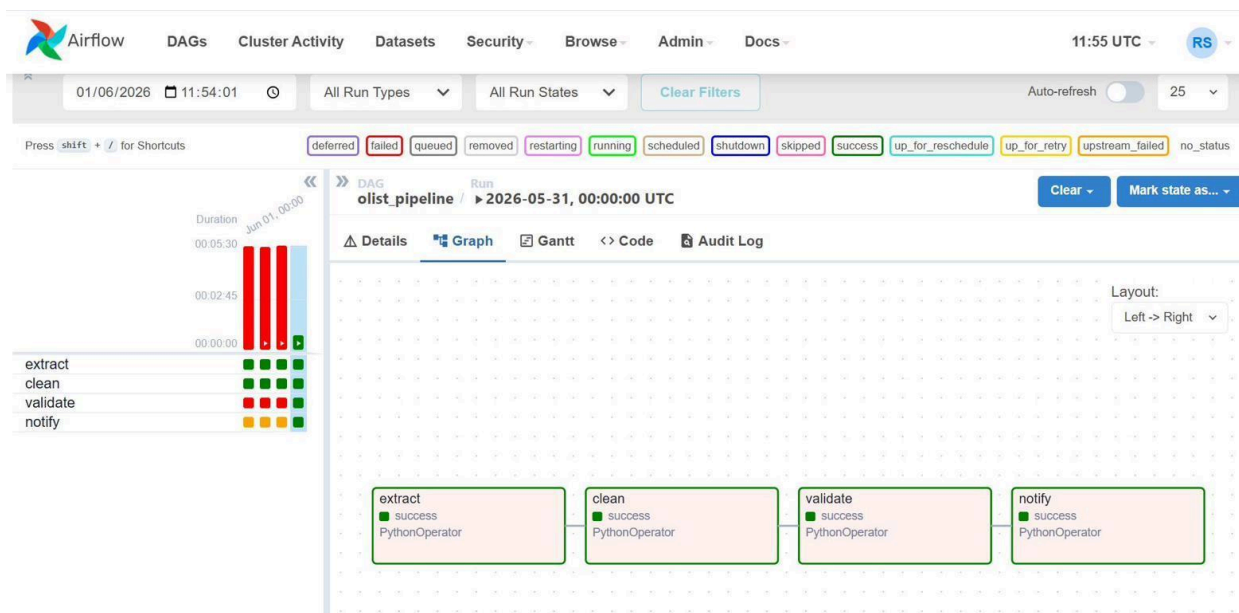


Figure 5: Airflow DAG `olist_pipeline` — all 4 tasks green (`extract`, `clean`, `validate`, `notify`)

Environment configuration in Windows.

Setting up Airflow on Windows requires WSL, as it does not run natively on Windows. Inside WSL, Python 3.14 was the system default, but Airflow 2.9.2 requires Python 3.11. A separate virtual environment was created with Python 3.11. Connecting WSL to Windows PostgreSQL required finding the Windows host IP from WSL's network configuration and whitelisting the WSL subnet in `pg_hba.conf`.

Data Quality Assurance encompassed three primary categories:

1. The method to handle NULL values was determined for each impacted field. For example, do you choose to: a) fill, b) remove (drop), or c) preserve NULLs?
2. Foreign Key Integrity checks were performed at the level of Python prior to any data being fed into PostgreSQL. Four distinct relationships (across all four fields) were evaluated against the data set.
3. Test Set Validation confirmed that there were ZERO broken references within all four integrity checks as evidence that the pipeline generates "clean" data from new inputs.

Test set validation results:

Check	Result	Status
Order items with invalid order_id	0	PASS
Orders with invalid customer_id	0	PASS
Order items with invalid product_id	0	PASS
Order items with invalid seller_id	0	PASS
Test orders row count	24,860 (expected ~24,860)	PASS

6. Results & Analytical Insights

The development set loaded 74,581 orders across all nine tables with zero constraint violations. The test set validation confirmed zero foreign key violations across all four integrity checks.

Row Count Summary

Table	Dev Set	Test Set	Full Dataset
Orders	74,581	24,860	99,441
Customers	74,581	24,860	99,441
order_items	84,439	28,143	112,650
Payments	77,931	26,060	103,886
Reviews	73,804	24,785	~98,589

Table	Dev Set	Test Set	Full Dataset
Products	32,951	32,951	32,951
Sellers	3,095	3,095	3,095
Geolocation	1,000,163	1,000,163	1,000,163
category_translation	74	74	74

Key Analytical Findings

- Forty percent of the orders were from São Paulo. This is consistent with its status as Brazil's largest economic center.
- The average rating per product was five stars. The overall level of customer satisfaction on the site appears to be very positive.
- The majority of ratings provided were five-star. However, there were many more one-star ratings than four- or two-star ratings.
- The number of order items is always greater than the total number of orders since each order can include more than one item. Similarly, the number of payments received is always greater than the number of orders, since some customers pay for their orders using multiple payment methods.
- There were at least 600 orders in the development set without a matching review.

The Power BI Dashboard includes three tabbed sections for analytical reporting: Overview (Total KPIs and Order Trend Analysis), Delivery Performance (On-Time vs. Late by State and Month), and Commerce & Reviews (Revenue by Category, Top Sellers, and Review Distribution).

7. Challenges & Recommendations

Technical Challenges Encountered

Challenge	Root Cause	Resolution
SSL certificate errors on pip install and Kaggle API	Our network SSL proxy intercepting HTTPS connections	Added trusted-host flags to bypass certificate verification
Python 3.14 incompatible with Airflow 2.9.2	System Python version too new for Airflow requirements	Created separate Python 3.11 virtual environment inside WSL
WSL could not connect to Windows PostgreSQL	localhost in WSL points to Linux, not Windows host	Found Windows host IP via /etc/resolv.conf, whitelisted WSL subnet in pg_hba.conf
Foreign key violations on product load	Products referenced category names not in the category translation table	Programmatically detected and added all missing categories before loading

Challenge	Root Cause	Resolution
Duplicate review IDs are causing primary key violations	Bug in Olist review submission system, same ID reused for multiple submissions	Applied drop duplicates, keeping the first occurrence

Recommendations for Improvement.

- Implement upset logic in your PostgreSQL database using the on conflict do nothing syntax so that incremental batch loads are handled without dropping and reloading the entire database (to handle larger datasets).
- Migrate raw data storage to cloud object storage such as AWS S3 so the pipeline does not rely on local file paths.
- Add quality checks in your DAG for Airflow to automate integrity checks that currently have to be done manually by running a notebook script between clean and validate tasks.

8. References & Supporting Material

Olist. (2018). Brazilian E-Commerce Public Dataset by Olist [Data set]. Kaggle.
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Apache Software Foundation. (2024). Apache Airflow documentation (Version 2.9.2).
<https://airflow.apache.org/docs/>

PostgreSQL Global Development Group. (2024). PostgreSQL 18 documentation.
<https://www.postgresql.org/docs/18/>

Microsoft. (2024). Power BI documentation. <https://learn.microsoft.com/en-us/power-bi/>

McKinney, W. (2022). Python for data analysis (3rd ed.). O'Reilly Media.

SQLAlchemy. (2024). SQLAlchemy documentation (Version 2.0). <https://docs.sqlalchemy.org/>